

Laboratório de Concorrência · Aula 10

Renan Eizo Miyazaki Takushi · RA 240382312 · Turma C

Gerado em 09/10/2026, 21:51:55 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação, uma tarefa precisa esperar a outra terminar algo para poder continuar. Ninguém disputa nada, só existe uma dependência de ordem. Exemplo: o garçom só leva o prato depois que o cozinheiro termina. Na competição, duas tarefas querem o mesmo recurso, que só pode ser usado por uma de cada vez. Exemplo: duas pessoas querendo usar o mesmo caixa eletrônico, e uma tem que esperar a outra sair.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2000000

1ª execução (n = 1.000.000)

1422363

2ª execução (n = 1.000.000)

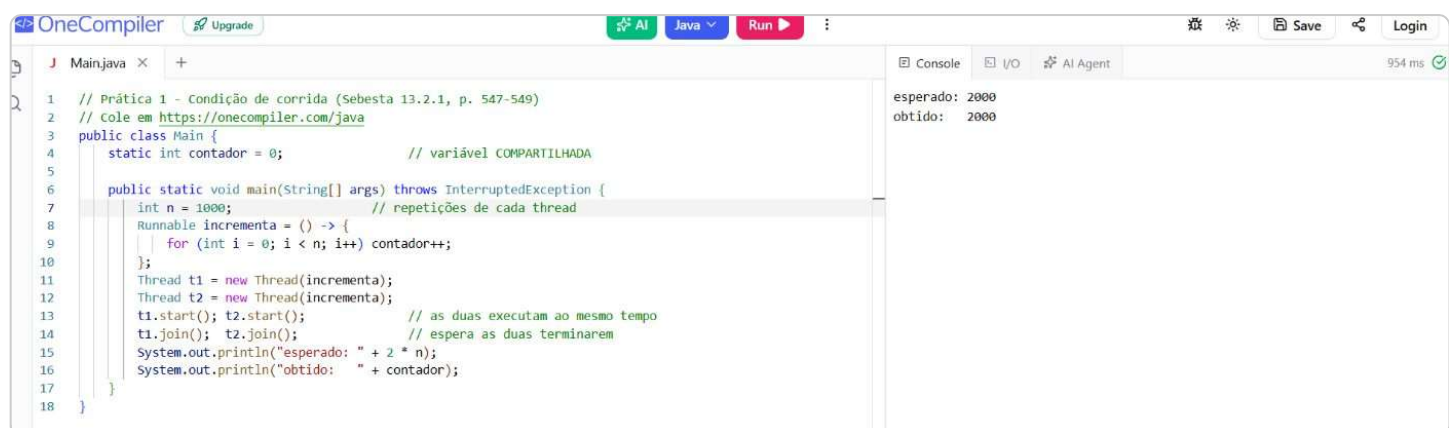
1032558

3ª execução (n = 1.000.000)

1045524

Saída do programa

não respondido



```
// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
// Cole em https://onecompiler.com/java
public class Main {
    static int contador = 0; // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1000; // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start(); // as duas executam ao mesmo tempo
        t1.join(); t2.join(); // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido: " + contador);
    }
}
```

Console: 954 ms ✓

esperado: 2000
obtido: 2000

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram. O esperado era 2.000.000, mas deu [seus valores], sempre menos que isso, e cada execução deu um número diferente. Isso acontece porque quem decide a ordem em que as threads executam é o escalonador, e ele intercala as duas de um jeito diferente a cada execução. Então a quantidade de somas perdidas também muda.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Supondo que o contador vale 10: a thread A lê 10, e logo depois a thread B também lê 10, porque A ainda não gravou. A soma 1 e fica com 11 no registrador, e B faz a mesma coisa. A grava 11 na memória, e B grava 11 por cima. As duas somaram, mas o contador só subiu 1, então a soma de uma delas se perdeu.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$ deu 2000, o valor certo. A conta é tão rápida que a primeira thread termina antes da segunda começar de verdade, então elas quase não se sobrepõem. O perigo é que nos testes, com pouco dado, o programa parece funcionar. Em produção, com muitos acessos ao mesmo tempo, o erro aparece de forma aleatória e é difícil de reproduzir e achar a causa.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

AABBAB → 6

ABAABB → 6

BBBAAA → 7

BBABAA → 4

Saída do programa

não respondido

```

3 # TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL 2) calcular 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19            passo[t] += 1
20    return total
21
22
23 print("AAABBB ->", executa("AAABBB"))
24 print("BBBAAA ->", executa("BBBAAA"))
25 print("ABABAB ->", executa("ABABAB"))
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = ""
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")

```

Console Output:

```

AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB

```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis e 4 resultados diferentes (4, 6, 7 e 8). Só uma ordem dá o resultado "correto" 8, a AAABBB. Ou seja, de 20 jeitos de executar, só 1 dá o que o livro considera certo.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBBA: A lê TOTAL = 3 e calcula 4 no registrador. Antes de A gravar, B lê TOTAL, que ainda é 3, calcula 6 e grava 6. Por último A grava o 4 dela por cima. A multiplicação de B foi apagada, como se nunca tivesse acontecido, porque A estava trabalhando com um valor desatualizado.

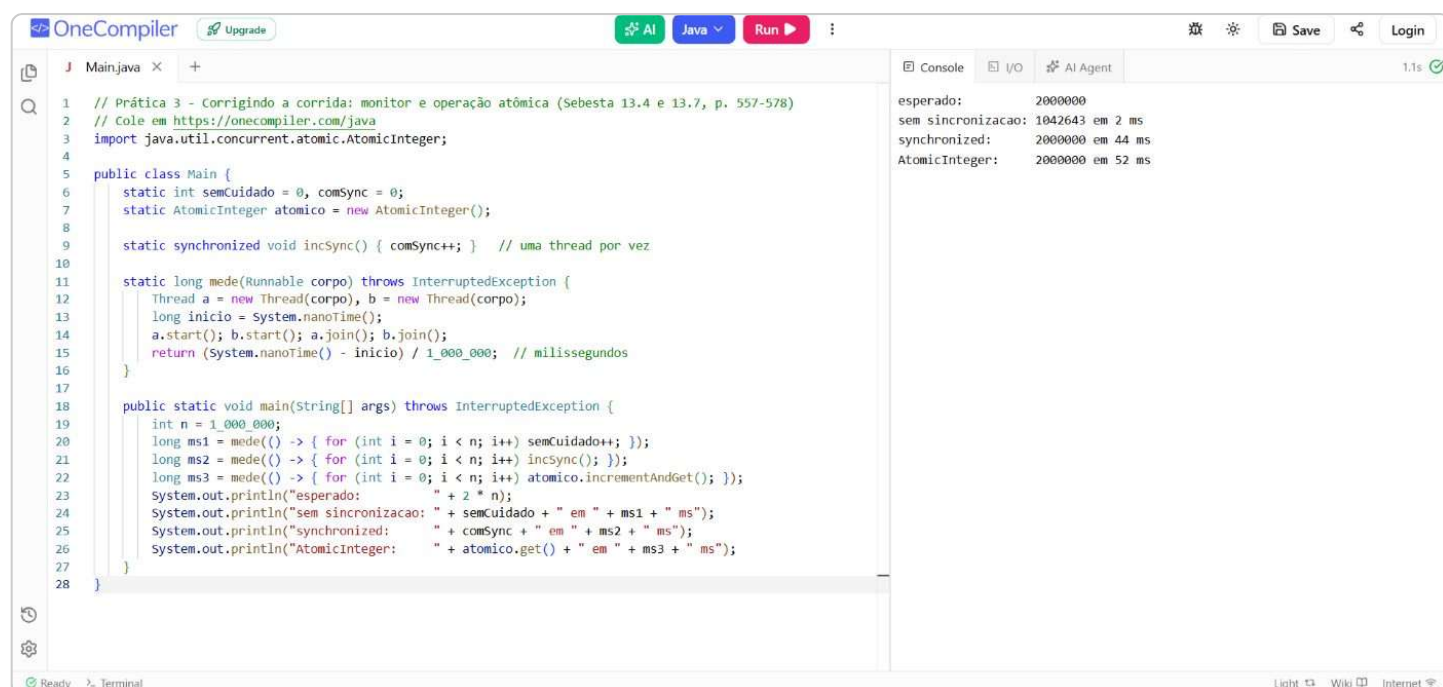
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Sincronização

Saída do programa

não respondido



```
// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

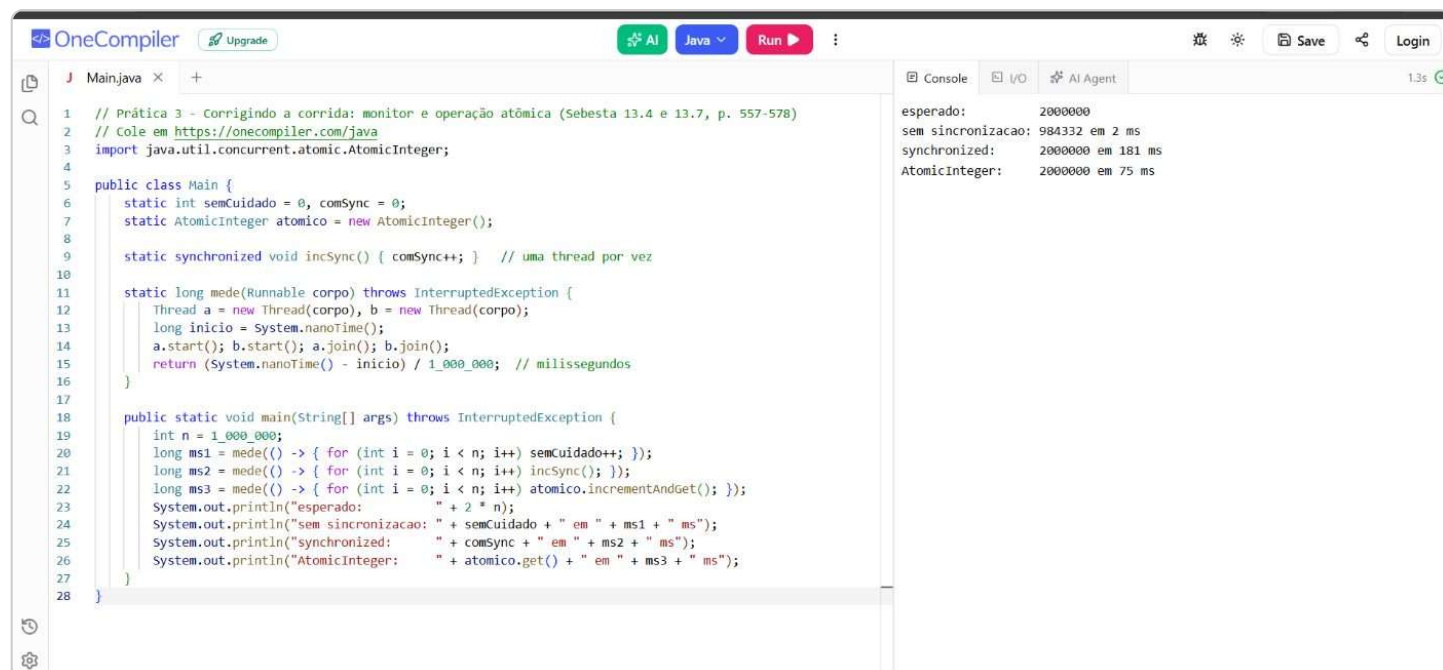
    static synchronized void incSync() { comSync++; } // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado: " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

Console Output:

Resultado	Tempo (ms)
esperado:	2000000
sem sincronizacao:	1042643 em 2 ms
synchronized:	2000000 em 44 ms
AtomicInteger:	2000000 em 52 ms



```
// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; } // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado: " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

Console Output:

Resultado	Tempo (ms)
esperado:	2000000
sem sincronizacao:	984332 em 2 ms
synchronized:	2000000 em 181 ms
AtomicInteger:	2000000 em 75 ms

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Só a versão sem sincronização errou. A correta é mais lenta porque as threads precisam pegar a trava e esperar uma pela outra.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, porque a transferência são duas operações e alguém pode ver o estado no meio delas. A transferência inteira precisa ser atômica, com uma trava nas duas contas ou uma transação.

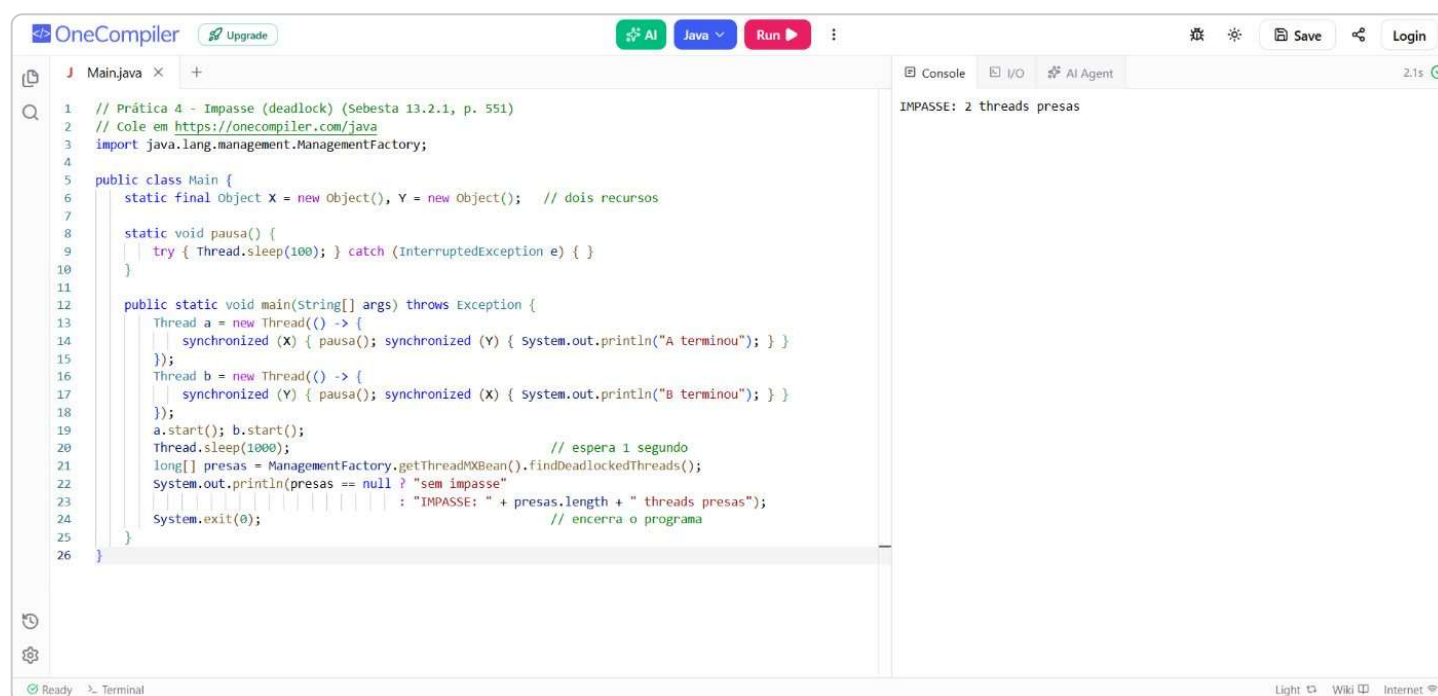
Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não

Saídas (antes e depois da correção)

não respondido



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

IMPASSE: 2 threads presas

```

1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }

```

Console output:

```

A terminou
B terminou
sem impasse

```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A segurava X e esperava Y, e B segurava Y e esperava X. Uma esperava a outra, para sempre.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Quem pega X consegue tudo e a outra espera sem segurar nada, então não forma o ciclo. Exemplo: duas transferências bancárias opostas ao mesmo tempo.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Vai dar erro

Tempo com 1 trabalhador

200

Tempo com 3 trabalhadores

600

Tempo com 6 trabalhadores

100

Saída do programa

não respondido

```

1  main.go
2
3  import (
4      "fmt"
5      "sync"
6      "time"
7  )
8
9
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

Console Output:

```

trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 6
trabalhador 3 fez a tarefa 4
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

```

```

1  // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2  // Cole em https://onecompiler.com/go
3  package main
4
5  import (
6      "fmt"
7      "sync"
8      "time"
9  )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33 }

```

Console Output:

```

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

```

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33 }

```

Console output:

```

trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
trabalhador 2 fez a tarefa 3
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

O tempo é $(6 \div \text{trabalhadores}) \times 100 \text{ ms}$, porque eles trabalham ao mesmo tempo. Não precisa de trava porque cada tarefa vai para um só trabalhador pelo canal, sem variável compartilhada.

Saídas (antes e depois da troca)

não respondido

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int) // canal SEM espaço (sem buffer)
9     ch <- 1 // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

Console output:

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2

```

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1) // canal SEM espaço (sem buffer)
9     ch <- 1 // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

Console output:

```

1

```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Sem espaço, o envio espera alguém receber, mas o recebimento está na linha de baixo, na mesma goroutine. Com espaço para 1, o valor fica guardado no canal e o programa segue.

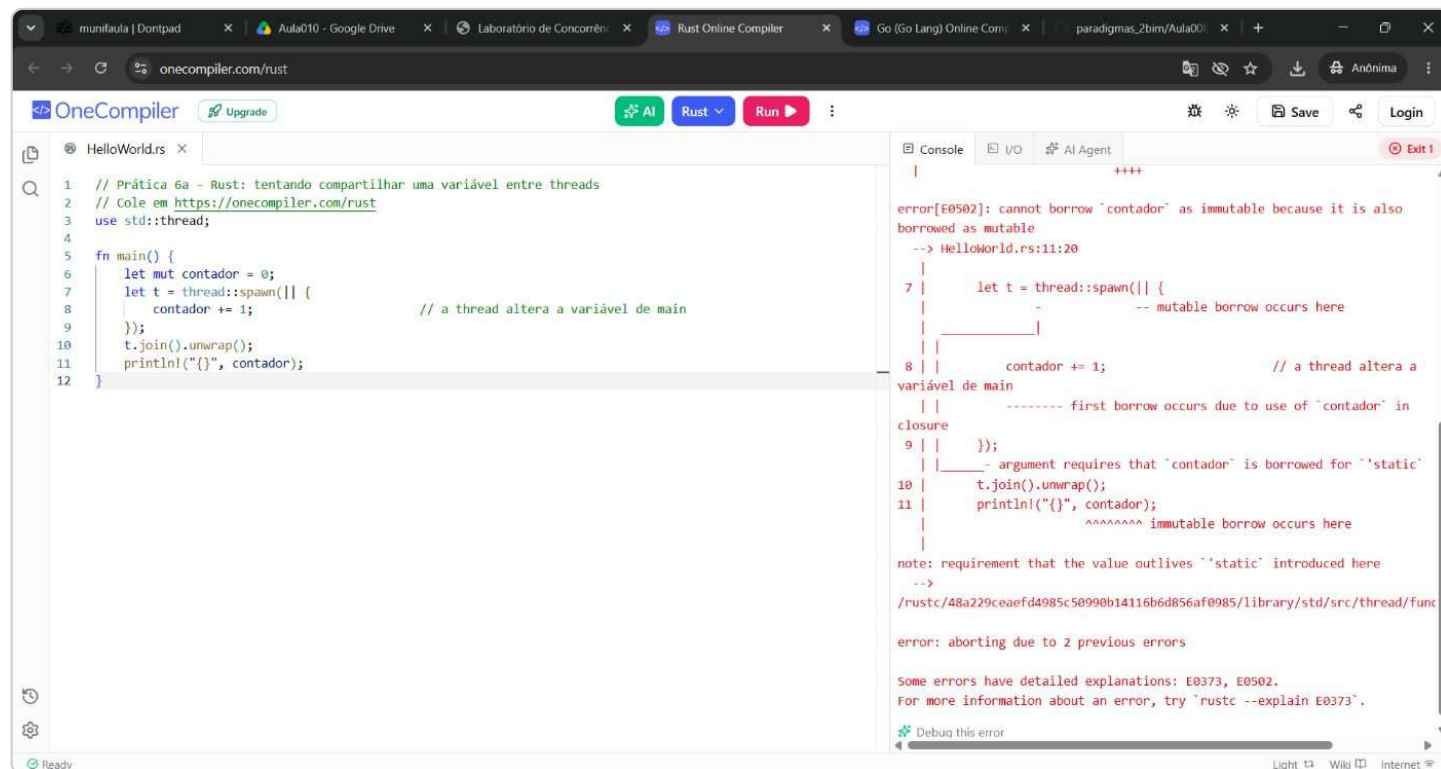
Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

não respondido

Saídas (o erro e o resultado com move)

não respondido



The screenshot shows the OneCompiler Rust IDE. The code in the editor is as follows:

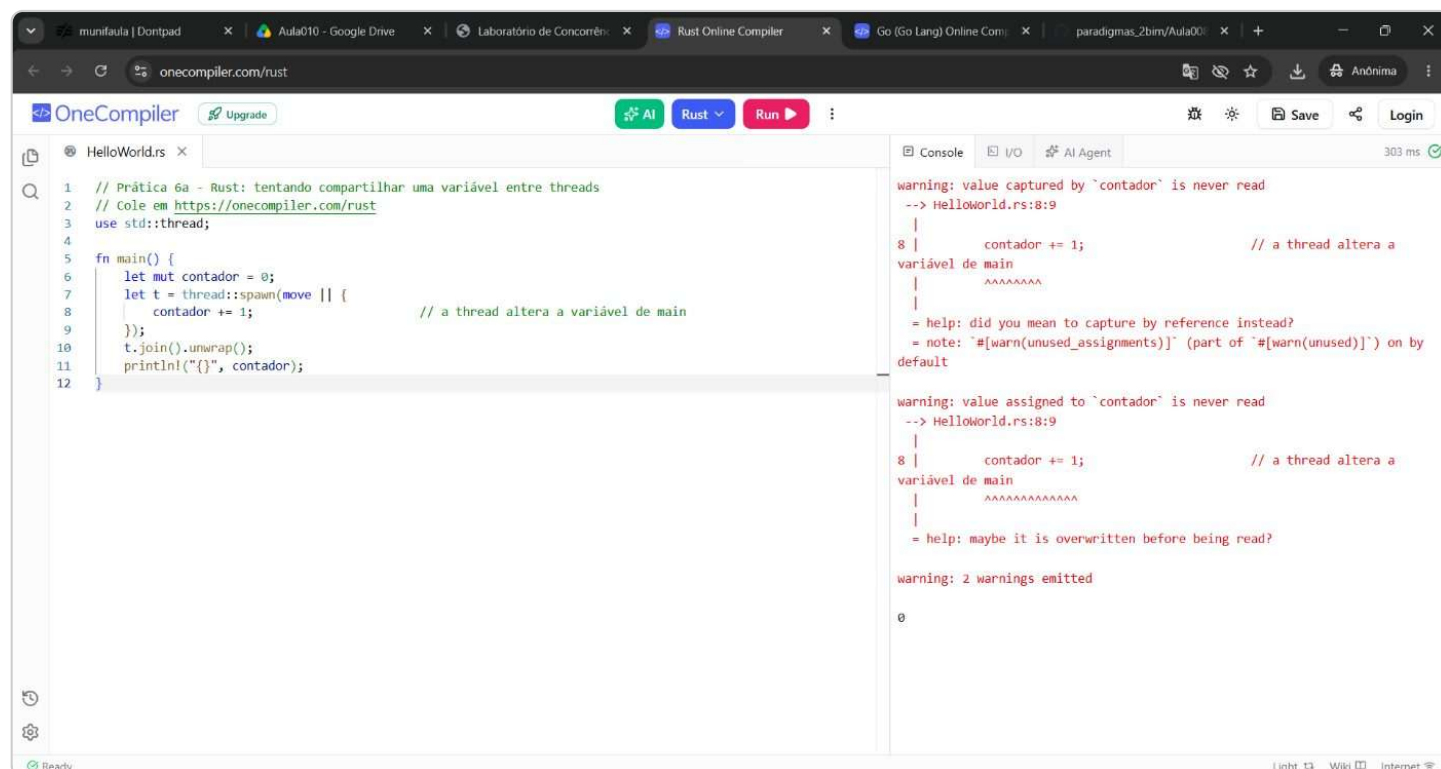
```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

The console on the right displays the following error:

```
error[E0502]: cannot borrow `contador` as immutable because it is also
borrowed as mutable
--> HelloWorld.rs:11:20
|
7 |         let t = thread::spawn(|| {
|         ~~~~~ mutable borrow occurs here
|
8 |         contador += 1; // a thread altera a
|         ~~~~~ first borrow occurs due to use of `contador` in
closure
9 |     });
|     ~ argument requires that `contador` is borrowed for `static`
10    t.join().unwrap();
11    println!("{}", contador);
|     ~~~~~ immutable borrow occurs here
|
note: requirement that the value outlives `static` introduced here
-->
/rustc/48a229ceaefd4985c50990b14116bd856af0985/library/std/src/thread/fun

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.
```



The screenshot shows the OneCompiler Rust IDE with the same code as the previous image, but with a change in the thread creation:

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(move || {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

The console on the right displays the following warnings:

```
warning: value captured by `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread altera a
|         ~~~~~
|
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by
default

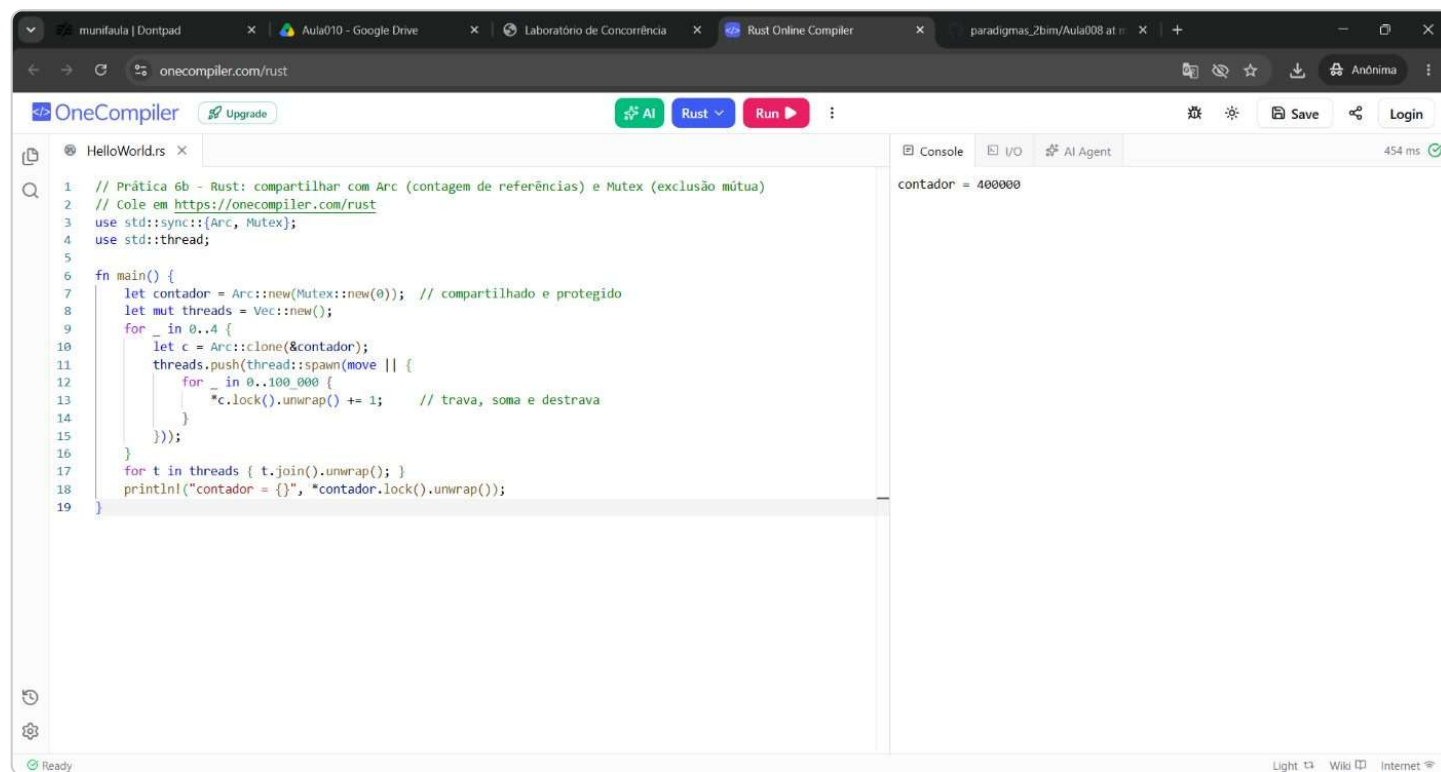
warning: value assigned to `contador` is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread altera a
|         ~~~~~
|
= help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0
```

Saída do programa

não respondido



```
1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }
```

contador = 400000

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move leva uma cópia do número para a thread. Ela soma na cópia, e o original continua 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro apareceu rodando, em silêncio, e em Rust apareceu ao compilar. A vantagem é mais segurança, e o custo é um código mais trabalhoso de escrever.

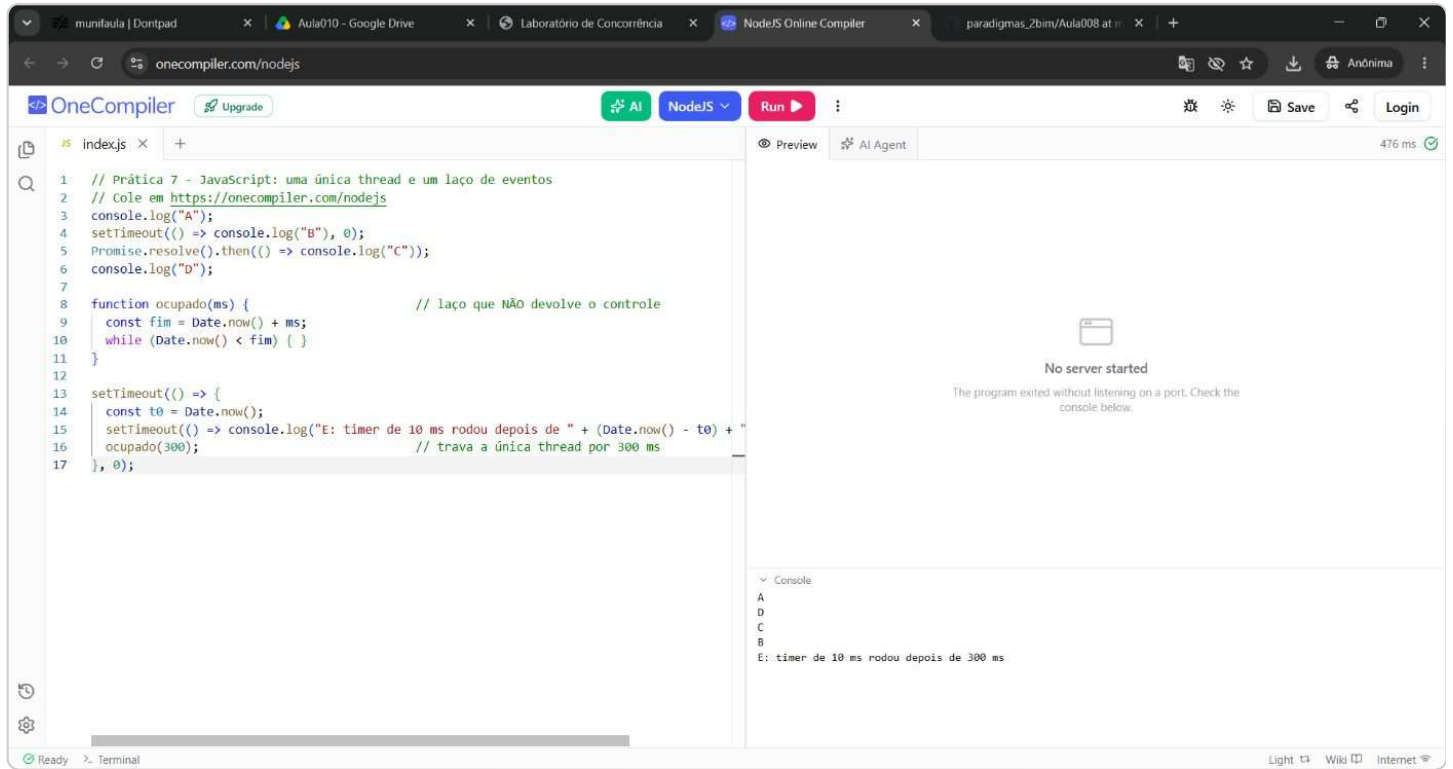
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, B, C

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. Primeiro roda o código normal, depois as promessas, e só depois os timers.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O laço ocupado travou a única thread por 300 ms, e o timer só roda quando ela fica livre. Num servidor, um cálculo pesado trava todos os usuários.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Condição de corrida: os dois veem 1 lugar e o ingresso é vendido duas vezes. Dá para evitar fazendo a verificação e a reserva numa operação atômica no banco de dados.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Escreva a sua. Um exemplo: “Me surpreendeu o código com $n = 1000$ funcionar e com 1.000.000 não.”